

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	N Adi narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	14-07-2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

1. Resume Information Extraction Using Python Regular Expressions

Aim

To develop a Python program that extracts candidate details from resumes using Regular Expressions.

Introduction

Resume Information Extraction helps automate the recruitment process by extracting details such as name, email, mobile number, technical skills, and years of experience. Python Regular Expressions (Regex) identify these patterns efficiently and generate a structured summary for easy candidate shortlisting.

Objective

- Shortlist eligible candidates.
- Extract name, email, phone, skills, experience.
- Generate summary.
- Shortlist candidates with Python and ≥ 2 years experience.

Algorithm

- Read resume.
- Extract fields using regex.
- Generate summary.
- Check eligibility.
- Display eligible candidates.

Software Requirements

Software	Version
Python	3.x
VS Code	Latest
re Module	Built-in

Features

- Resume parsing
- Skill detection
- Experience extraction
- Candidate shortlisting

Python Program

```
import re
# Sample resumes
resumes = [
    """
    John Smith
    Email: johnsmith@gmail.com
    Phone: +91-9876543210
```

```
"""
```

```
Priya Sharma
```

```
Email: priya@gmail.com
```

```
Mobile: 9123456789
```

```
Skills: Java, SQL
```

```
Experience: 1 year
```

```
""",
```

```
"""
```

```
Rahul Kumar
```

```
Email: rahul123@yahoo.com
```

```
Contact: 9876512345
```

```
Skills: Python, Java, SQL
```

```
Experience: 5 years
```

```
"""
```

```
]
```

```
# Technical skills list
```

```
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
```

```
def extract_information(text):
```

```
    # Name (first line)
```

```
    name = text.strip().split('\n')[0]
```

```
    # Email
```

```
    email = re.findall(r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}', text)
```

```
    # Phone Number
```

```
    phone = re.findall(r'(\+91[- ])?[6-9]\d{9}', text)
```

```
    # Skills
```

```
    skills_found = []
```

```
    for skill in skill_list:
```

```
        if re.search(skill, text, re.IGNORECASE):
```

```
            skills_found.append(skill)
```

```
    # Experience
```

```

experience = re.search(r'(\d+)\s+year', text, re.IGNORECASE)
years = int(experience.group(1)) if experience else 0
return {
    "Name": name,
    "Email": email[0] if email else "Not Found",
    "Phone": phone[0] if phone else "Not Found",

    "Skills": skills_found,
    "Experience": years
}
eligible = []
print("----- Candidate Summary -----")
for resume in resumes:
    candidate = extract_information(resume)
    print("Name :", candidate["Name"])
    print("Email :", candidate["Email"])
    print("Phone :", candidate["Phone"])
    print("Skills :", " , ".join(candidate["Skills"]))
    print("Experience :", candidate["Experience"], "Years")
    if candidate["Experience"] >= 2 and "Python" in candidate["Skills"]:
        eligible.append(candidate)
    print("-----")
print("\nEligible Candidates\n")
for person in eligible:
    print(person["Name"], "-", person["Experience"], "Years")

```

•

Sample Output

Field	Value
Name	Rahul Sharma
Email	rahul@gmail.com
Skills	Python, SQL
Experience	3 Years

Status	Eligible
--------	----------

Conclusion

The system automates resume screening, reduces manual effort, and helps recruiters identify suitable candidates quickly.

2. Product Search System Using Python Regular Expressions

Aim

To develop a Product Search System using Python Regular Expressions.

Introduction

The Product Search System allows users to search products using exact, prefix, suffix, partial, and case-insensitive matching. Regular Expressions improve search flexibility and provide faster results.

Objective

- Perform different types of product searches.
- Display matching products.
- Show total search results.

Software Requirements

Software	Version
Python	3.x
VS Code	Latest
re Module	Built-in

Features

- Exact search
- Prefix search
- Suffix search
- Partial search
- Case-insensitive search

Python Program

```
import re
products=["Python Book","Java Programming","SQL Guide","Machine Learning Kit",
"NLP Toolkit","Python Mug","Smart Phone","Phone Cover","Laptop Stand"]
```

```
def search_products(keyword, mode):
```

```

if mode=="exact":
    pattern=f"^{re.escape(keyword)}$"
elif mode=="prefix":
    pattern=f"^{re.escape(keyword)}"
elif mode=="suffix":
    pattern=f"{re.escape(keyword)}$"
elif mode=="partial":
    pattern=re.escape(keyword)
result=[p for p in products if re.search(pattern,p,re.IGNORECASE)]
print("Matches:",result)
print("Total:",len(result))

search_products("Python","prefix")
search_products("Guide","suffix")
search_products("Phone","partial")
search_products("python book","exact")

```

Sample Output

Search	Result
Phone	Samsung Phone, iPhone
Total Matches	2

Conclusion

The system provides fast and flexible product searching using Regular Expressions, improving the user experience.

3. University Registration Validation System Using Python Regular Expressions

Aim

To validate student registration details using Python Regular Expressions.

Introduction

The University Registration Validation System checks student information such as register number, email, course code, semester, and mobile number. Regular Expressions ensure that all details follow the required format before registration.

Objective

- Validate register number.
- Validate email address.
- Validate course code.
- Validate semester.
- Validate mobile number.

Software Requirements

Software	Version
Python	3.x
VS Code	Latest

re Module	Built-in
-----------	----------

Features

- Register number validation
- Email validation
- Course code validation
- Mobile number validation
- Registration status report

Python Program

```
import re
reg_no="23CSE1234"
email="student@university.edu"
course="CSE301"
semester="Semester 5"
mobile="9876543210"

print("Register No:",bool(re.fullmatch(r"\d{2}[A-Z]{3}\d{4}",reg_no)))
print("Email:",bool(re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu",email)))
print("Course:",bool(re.fullmatch(r"[A-Z]{3}\d{3}",course)))
print("Semester:",bool(re.fullmatch(r"Semester\s([1-8])",semester)))
print("Mobile:",bool(re.fullmatch(r"[6-9]\d{9}",mobile)))

if all([
re.fullmatch(r"\d{2}[A-Z]{3}\d{4}",reg_no),
re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu",email),
re.fullmatch(r"[A-Z]{3}\d{3}",course),
re.fullmatch(r"Semester\s([1-8])",semester),
re.fullmatch(r"[6-9]\d{9}",mobile)]):
    print("Registration Successful")
else:
    print("Registration Failed")
    ●
```

Sample Output

Field	Status
Register Number	Valid
Email	Valid
Course Code	Valid
Mobile Number	Valid
Registration	Successful

Conclusion

The system validates student details accurately, reduces registration errors, and ensures that only valid information is accepted.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____